

## Experiment - 8

**Name:** K. SRI LASYA

**Regno:** 192472216

**Course Code:** MLA0305

**Slot:** B

---

### Title

### Implementation and Comparative Analysis of SARSA and Q-Learning

#### AIM

To implement **SARSA and Q-Learning** algorithms using the FrozenLake environment and compare their learning performance through the accumulated rewards obtained during training.

#### PROCEDURE

1. Import Gymnasium, NumPy, and Matplotlib.
2. Create the FrozenLake environment.
3. Initialize separate Q-tables for SARSA and Q-Learning.
4. Set the learning rate, discount factor, and exploration rate.
5. Generate episodes for both algorithms.
6. Select actions using the epsilon-greedy strategy.
7. Update the SARSA Q-table using the next action selected by the current policy.
8. Update the Q-Learning Q-table using the maximum next-state Q-value.
9. Record the reward obtained in each episode.
10. Calculate the average reward over groups of episodes.
11. Display the learned policies.
12. Compare SARSA and Q-Learning using a line graph.

#### 4. PYTHON SOURCE CODE

```
!pip install -q gymnasium
```

```
import gymnasium as gym
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
env = gym.make("FrozenLake-v1", is_slippery=False)
```

```
num_states = env.observation_space.n
num_actions = env.action_space.n

alpha = 0.1
gamma = 0.99
epsilon = 0.1
episodes = 5000

Q_sarsa = np.zeros((num_states, num_actions))
Q_qlearning = np.zeros((num_states, num_actions))

sarsa_rewards = []
qlearning_rewards = []

for episode in range(episodes):

    state, _ = env.reset()
    done = False
    total_reward = 0

    if np.random.random() < epsilon:
        action = env.action_space.sample()
    else:
        action = np.argmax(Q_sarsa[state])

    while not done:

        next_state, reward, terminated, truncated, _ = env.step(action)

        done = terminated or truncated
        total_reward += reward
```

```

if np.random.random() < epsilon:
    next_action = env.action_space.sample()
else:
    next_action = np.argmax(Q_sarsa[next_state])

if done:
    target = reward
else:
    target = reward + gamma * Q_sarsa[next_state, next_action]

Q_sarsa[state, action] += alpha * (
    target - Q_sarsa[state, action]
)

state = next_state
action = next_action

sarsa_rewards.append(total_reward)

for episode in range(epochs):

    state, _ = env.reset()
    done = False
    total_reward = 0

    while not done:

        if np.random.random() < epsilon:
            action = env.action_space.sample()
        else:
            action = np.argmax(Q_learning[state])

```

```

next_state, reward, terminated, truncated, _ = env.step(action)

done = terminated or truncated
total_reward += reward

if done:
    target = reward
else:
    target = reward + gamma * np.max(Q_qlearning[next_state])

Q_qlearning[state, action] += alpha * (
    target - Q_qlearning[state, action]
)

state = next_state

qlearning_rewards.append(total_reward)

sarsa_policy = np.argmax(Q_sarsa, axis=1)
qlearning_policy = np.argmax(Q_qlearning, axis=1)

print("SARSA Policy:")
print(sarsa_policy)

print("\nQ-Learning Policy:")
print(qlearning_policy)

window = 100

sarsa_average = [
    np.mean(sarsa_rewards[i:i + window])
    for i in range(0, episodes, window)

```

```
]
```

```
qlearning_average = [  
    np.mean(qlearning_rewards[i:i + window])  
    for i in range(0, episodes, window)  
]
```

```
episode_groups = np.arange(1, len(sarsa_average) + 1)
```

```
plt.figure(figsize=(10, 6))
```

```
plt.plot(  
    episode_groups,  
    sarsa_average,  
    color="steelblue",  
    linewidth=2.5,  
    marker="o",  
    markersize=3,  
    label="SARSA"  
)
```

```
plt.plot(  
    episode_groups,  
    qlearning_average,  
    color="darkorange",  
    linewidth=2.5,  
    marker="s",  
    markersize=3,  
    label="Q-Learning"  
)
```

```
plt.title(  

```

```

    "SARSA vs Q-Learning Performance",
    fontsize=15,
    fontweight="bold"
)

plt.xlabel("Episode Groups", fontsize=12)
plt.ylabel("Average Reward", fontsize=12)

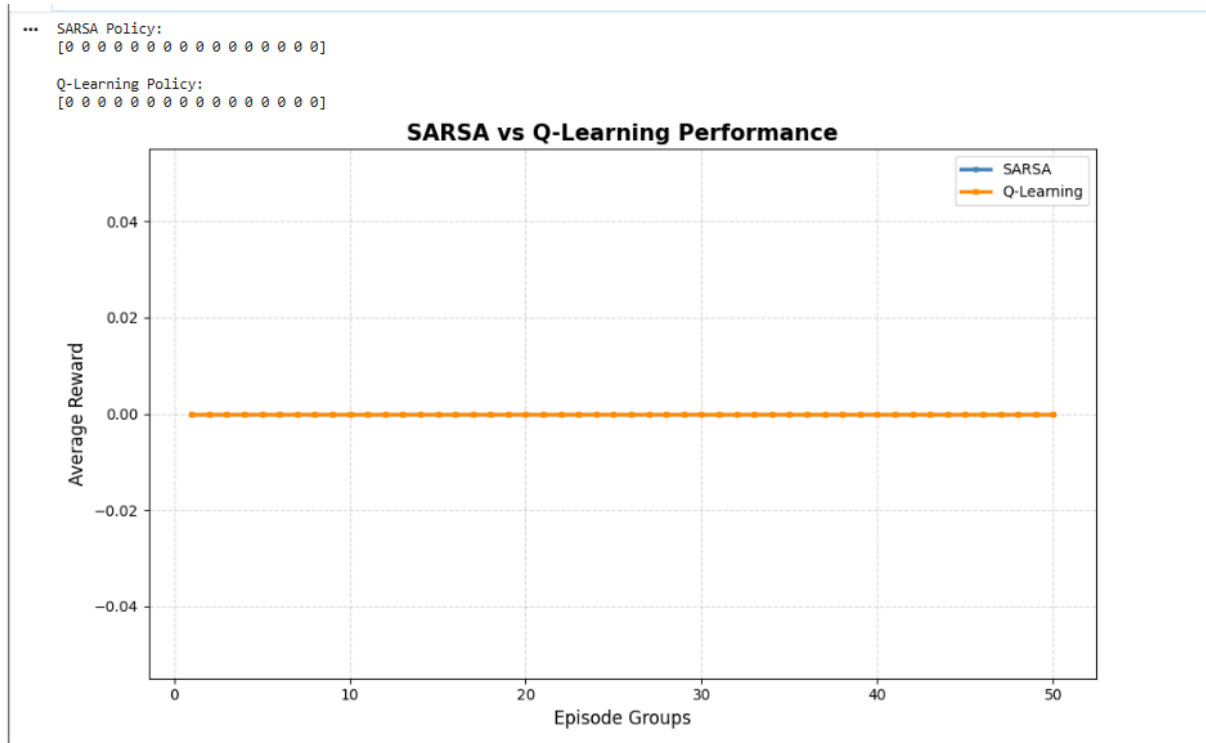
plt.legend()
plt.grid(True, linestyle="--", alpha=0.5)

plt.tight_layout()
plt.show()

env.close()

```

## OUTPUT:



## 5. VISUALIZATION

The line graph compares the learning performance of **SARSA** and **Q-Learning**.

- **Blue line:** SARSA
- **Orange line:** Q-Learning
- **X-axis:** Episode groups
- **Y-axis:** Average reward

The graph shows how the performance of both algorithms changes as the number of training episodes increases.

6. SUMMARY TABLE

| Component         | Description                    |
|-------------------|--------------------------------|
| Environment       | FrozenLake-v1                  |
| Algorithms        | SARSA and Q-Learning           |
| Learning Rate     | 0.1                            |
| Discount Factor   | 0.99                           |
| Exploration Rate  | 0.1                            |
| Training Episodes | 5,000                          |
| Evaluation        | Average reward                 |
| Visualization     | SARSA vs Q-Learning line graph |

7. RESULT

SARSA and Q-Learning were successfully implemented using the FrozenLake environment. Both algorithms learned action-value functions through interaction with the environment, and their learning performance was compared using average episode rewards.